

# Write Basic Neural Network Using NumPy, Keras, and PyTorch

Assignment report documenting repository structure, implementation variants, execution evidence, and video walkthrough mapping.

|                 |                                                                                                                                                                                     |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Student Name    | Shilpa Yelkur Ramakrishnaiah                                                                                                                                                        |
| Student ID      | 019151782                                                                                                                                                                           |
| Repository      | <a href="https://github.com/ShilpaYR/Writing-basic-neural-network-using-numpy-keras-pytorch">https://github.com/ShilpaYR/Writing-basic-neural-network-using-numpy-keras-pytorch</a> |
| Submission Type | Public GitHub URL with executed Colab notebooks, walkthrough videos, and README documentation                                                                                       |

This report summarizes the complete submission for the neural network assignment. The repository contains eight Jupyter/Colab notebooks covering NumPy-style manual backpropagation, three PyTorch variants, and four TensorFlow variants for nonlinear regression with three input variables. Each implementation follows the same core task: generate synthetic nonlinear data, visualize the 4D-style data distribution using PCA, train a 3-hidden-layer network, and evaluate the final model with regression metrics and output plots.

## 1. Assignment Coverage and Deliverables

The submission addresses the professor’s requirements by providing a public GitHub repository, clean notebook naming, executed outputs, video walkthroughs, and implementation diversity across frameworks and abstraction levels. The notebooks collectively demonstrate manual parameter handling, manual backpropagation, framework autograd, built-in module usage, functional APIs, model subclassing, and Lightning-based training orchestration.

| Item               | Deliverable                                    | Status    | Evidence         |
|--------------------|------------------------------------------------|-----------|------------------|
| Repository         | Public GitHub repo with all notebooks          | Completed | Repo listing     |
| Documentation      | README with notebook and video links           | Completed | README section   |
| Walkthrough videos | YouTube recordings covering all supplied demos | Completed | YouTube links    |
| Executed notebooks | Outputs visible inside each notebook           | Completed | Notebook outputs |

|                |                                                           |           |             |
|----------------|-----------------------------------------------------------|-----------|-------------|
| Model variants | NumPy, PyTorch, Lightning, and TensorFlow implementations | Completed | 8 notebooks |
|----------------|-----------------------------------------------------------|-----------|-------------|

## 2. Repository Inventory

| Notebook                                         | Framework                     | Primary implementation style               |
|--------------------------------------------------|-------------------------------|--------------------------------------------|
| A_numpy_style_manual_backprop (1).ipynb          | TensorFlow ops + manual logic | NumPy-style manual backprop with tf.einsum |
| B_pytorch_scratch_no_builtin_layers_V2 (1).ipynb | PyTorch                       | Raw trainable tensors, no built-in layers  |
| C_pytorch_class_based_builtin (1).ipynb          | PyTorch                       | nn.Module class-based network              |
| D_pytorch_lightning (1).ipynb                    | PyTorch Lightning             | LightningModule with trainer workflow      |
| E1_tensorflow_low_level (1).ipynb                | TensorFlow                    | Low-level custom network and GradientTape  |
| E2_tensorflow_builtin_layers (1).ipynb           | TensorFlow                    | Sequential / built-in Dense layers         |
| E3_tensorflow_functional_api (1).ipynb           | TensorFlow                    | Functional API graph definition            |
| E4_tensorflow_high_level_subclassing (1).ipynb   | TensorFlow                    | Model subclassing / high-level API         |

## 3. Video Walkthrough Mapping

Six YouTube links were provided. One video is titled for a combined PyTorch walkthrough and covers notebooks B, C, and D together. The remaining links map individually to A and E1-E4.

| Notebook(s)                                                                                      | Video title / scope                         | Link                                                                    |
|--------------------------------------------------------------------------------------------------|---------------------------------------------|-------------------------------------------------------------------------|
| A_numpy_style_manual_backprop                                                                    | A_numpy_style_manual_backprop Demo          | <a href="https://youtu.be/jx4tdm7OrFI">https://youtu.be/jx4tdm7OrFI</a> |
| B_pytorch_scratch_no_builtin_layers_V2;<br>C_pytorch_class_based_builtin;<br>D_pytorch_lightning | Combined PyTorch walkthrough covering B/C/D | <a href="https://youtu.be/H7VnsYx73RY">https://youtu.be/H7VnsYx73RY</a> |

|                                      |                                                 |                                                                         |
|--------------------------------------|-------------------------------------------------|-------------------------------------------------------------------------|
| E1_tensorflow_low_level              | E1_tensorflow_low_level demo                    | <a href="https://youtu.be/sS1-PAO4otQ">https://youtu.be/sS1-PAO4otQ</a> |
| E2_tensorflow_built_in_layers        | E2_tensorflow_built_in_layers                   | <a href="https://youtu.be/bSf2RFEcYuU">https://youtu.be/bSf2RFEcYuU</a> |
| E3_tensorflow_functional_api         | E3_tensorflow_functional_api                    | <a href="https://youtu.be/WupTljZCzh4">https://youtu.be/WupTljZCzh4</a> |
| E4_tensorflow_high_level_subclassing | E4_tensorflow_high_level_subclassing Demo Video | <a href="https://youtu.be/29ZadO7YeRs">https://youtu.be/29ZadO7YeRs</a> |

## 4. Technical Design Summary

**Synthetic data:** All notebooks use a three-variable nonlinear equation with additive noise, matching the assignment's requirement to extend beyond the two-variable classroom example.

**Network depth:** Every implementation uses a 3-hidden-layer neural network rather than a shallow 1- or 2-layer baseline.

**Visualization:** Each notebook includes a 4D-style visualization where PCA projects the three inputs into 3D and color encodes the target output.

**Training evidence:** The notebooks print dataset sizes, training progress, loss curves, and final evaluation metrics.

**Comparative value:** The repository demonstrates the same problem across low-level and high-level APIs, making the conceptual differences between frameworks easy to compare.

## 5. Notebook-by-Notebook Discussion

### A. NumPy-style Manual Backpropagation

This notebook demonstrates the most explicit learning pipeline. Parameters are initialized manually, ReLU and its derivative are defined explicitly, the forward pass is written step by step, and gradient propagation is carried out manually using the chain rule. The notebook also uses `tf.einsum` for linear transformations, satisfying the professor's note about replacing ordinary matrix multiplication. This version is valuable because it exposes the mathematical mechanics of hidden-layer learning, parameter gradients, and manual updates.

### B. PyTorch from Scratch without Built-in Layers

This notebook implements the network using raw PyTorch tensors with `requires_grad=True` instead of `nn.Linear` or other built-in layer abstractions. Forward propagation is coded manually, while PyTorch `autograd` handles gradient computation after `loss.backward()`. The update step is still written manually inside a `no_grad` block, which keeps the implementation close to the spirit of the scratch requirement while leveraging PyTorch's differentiation engine.

### C. PyTorch Class-Based Built-in Network

This version moves from raw tensors to a clean `nn.Module` architecture. The model structure is encapsulated in a reusable class, hidden layers are defined through built-in `Linear` modules, and optimization is handled through standard PyTorch loss and optimizer objects. This notebook demonstrates

the practical transition from educational scratch code to maintainable model code that follows common deep learning engineering patterns.

### D. PyTorch Lightning

This notebook lifts the PyTorch solution one level higher by placing the architecture, training logic, and validation logic inside a LightningModule. Data loaders are used for batch handling, and the Trainer object manages the epoch loop, optimizer steps, and validation flow. The result is the cleanest PyTorch implementation from a software-structure perspective and helps show how training boilerplate can be reduced in modern workflows.

### E1. TensorFlow Low-Level

The first TensorFlow variant defines trainable weights and biases directly and uses GradientTape to compute derivatives. Like the manual notebook, it exposes the internals of optimization, but now inside TensorFlow’s low-level execution model. This notebook is especially important because it demonstrates conceptual equivalence between the scratch-style PyTorch path and TensorFlow’s low-level training approach.

### E2. TensorFlow Built-in Layers

This notebook uses TensorFlow’s built-in Dense layers in a straightforward Sequential-style workflow. It is concise, readable, and suitable for demonstrating how rapidly the same nonlinear regression task can be implemented once the framework manages the parameter registration and training machinery.

### E3. TensorFlow Functional API

This notebook expresses the network as an explicit computation graph from input tensors to output tensors. The Functional API variant is useful when models need clearer graph structure, branching, or future extension. It provides a middle ground between the simplicity of Sequential and the flexibility of subclassing.

### E4. TensorFlow High-Level Subclassing

The final TensorFlow notebook defines a custom model class that inherits from tf.keras.Model. This approach offers more architectural flexibility and a strongly object-oriented organization while still benefiting from TensorFlow’s compile and fit workflow. It serves as the most customizable high-level TensorFlow implementation in the repository.

## 6. Comparative Performance Summary

The executed notebook outputs show that the higher-level PyTorch and TensorFlow implementations converge very strongly on this synthetic task, with  $R^2$  values near 0.99. The manual NumPy-style and scratch PyTorch versions remain educationally valuable, but their final error is larger because they rely on more manual training logic and less optimized implementation flow.

| Notebook                      | MSE | RMSE   | MAE    | $R^2$  | Interpretation              |
|-------------------------------|-----|--------|--------|--------|-----------------------------|
| A_numpy_style_manual_backprop | -   | 0.4299 | 0.3234 | 0.8550 | Manual educational baseline |

|                                        |          |          |          |          |                           |
|----------------------------------------|----------|----------|----------|----------|---------------------------|
| B_pytorch_scratch_no_builtin_layers_V2 | 0.342220 | 0.584996 | 0.460233 | 0.731524 | Scratch implementation    |
| C_pytorch_class_based_builtin          | 0.009611 | 0.098035 | 0.078273 | 0.992460 | Strong generalization     |
| D_pytorch_lightning                    | 0.010867 | 0.104243 | 0.085313 | 0.991475 | Clean high-level training |
| E1_tensorflow_low_level                | 0.011327 | 0.106428 | 0.087356 | 0.991114 | Low-level TF success      |
| E2_tensorflow_builtin_layers           | 0.016750 | 0.129423 | 0.106300 | 0.986859 | Good compact baseline     |
| E3_tensorflow_functional_api           | 0.014833 | 0.121792 | 0.095794 | 0.988363 | Flexible graph API        |
| E4_tensorflow_high_level_subclassing   | 0.010496 | 0.102451 | 0.083725 | 0.991766 | Best TF result here       |

## 7. Evidence of Assignment Alignment

| Professor requirement           | How the submission addresses it                                                             |
|---------------------------------|---------------------------------------------------------------------------------------------|
| Three-layer deep network        | All notebooks implement three hidden layers rather than a shallow 1- or 2-layer structure.  |
| Nonlinear regression            | Every implementation uses a synthetic nonlinear target function with three variables.       |
| Manual backprop and chain rule  | Notebook A explicitly implements the backward pass and parameter updates by hand.           |
| TensorFlow einsum note          | Notebook A uses tf.einsum in place of ordinary matrix multiplication for the linear layers. |
| PyTorch without built-in layers | Notebook B uses raw parameter tensors instead of module-level layer objects.                |
| PyTorch built-in class version  | Notebook C uses nn.Module and built-in layers.                                              |
| PyTorch Lightning variant       | Notebook D provides the Lightning implementation.                                           |

|                              |                                                                                               |
|------------------------------|-----------------------------------------------------------------------------------------------|
| TensorFlow variants          | E1 through E4 cover low-level, built-in layers, Functional API, and subclassing.              |
| Video walkthroughs in README | The supplied YouTube links are mapped into the README and summarized in this report.          |
| Public clean documentation   | The repository contains consistently named notebooks and is prepared for a structured README. |

## 8. Overall Evaluation

This submission shows both conceptual depth and practical breadth. It begins with a fully manual formulation of learning, progresses through increasingly structured PyTorch workflows, and ends with multiple TensorFlow abstractions. Because the repository uses the same nonlinear regression task across all variants, it becomes easy to compare implementation style, code structure, training convenience, and final performance. The inclusion of executed outputs, loss curves, regression metrics, repository organization, and walkthrough videos gives the submission a complete end-to-end character rather than a purely code-only presentation.

## 9. Conclusion

In conclusion, the repository demonstrates the requested neural network assignment in a comprehensive and well-documented way. It covers the professor's framework requirements, provides visual evidence of execution, and includes walkthrough videos that support code explanation and reproducibility. The strongest-performing variants are the structured PyTorch and TensorFlow implementations, while the scratch notebooks provide the educational foundation needed to explain backpropagation and parameter learning in detail.